Healthy Lifestyle Assistant

Multi-Agent System for Fitness, Nutrition and Daily Habits

Дисциплини: Интелигентни системи · Онтологично инженерство · Интелигентни софтуерни архитектури · Практикум
Автор: Даниел Златанов фн: 2501697001
Специалност: Магистър „Софтуерни технологии“ със специализация „Системи с изкуствен интелект“
Университет: Пловдивски университет «Паисий Хилендарски»
Година: 2025/2026

1. Резюме

Healthy Lifestyle Assistant (HLA) е интелигентна multi-agent система за персонализирани препоръки относно хранене, физическа активност и ежедневни навици. Проектът е разработен в рамките на магистърската програма „Системи с изкуствен интелект“ и интегрира знания от четири учебни дисциплини.
Системата комбинира Spring Boot web приложение, JADE agent platform с FIPA ACL комуникация, OWL онтология с Apache Jena reasoning (SPARQL) и H2 релационна база данни. Специализирани агенти — NutritionAgent и FitnessAgent — извличат препоръки от онтологията чрез свойствата mealSuitableForGoal и exerciseSuitableForGoal. CoordinatorAgent orchestration-ва ACL flow-а (включително HABITS и GENERAL), а GatewayAgent и AgentRequestQueue осигуряват bridge между Spring HTTP threads и JADE agent threads.
Ключов design principle е **hybrid storage**: статичното domain knowledge (храни, упражнения, цели) живее в OWL; динамичните данни (профил, daily logs, ACL audit) — в H2. **PersonalizedAdviceService** комбинира profile, последен daily log и query text за по-динамични (но не conversational) agent отговори. Препоръките са образователни и не заменят медицински съвет.

2. Въведение

2.1. Мотивация

Съвременните lifestyle приложения често използват hard-coded правила, които не обясняват *защо* дадена препоръка е подходяща и трудно се разширяват. HLA решава това чрез:

- **Онтология** — формален, машинно обработваем domain model;
- **Multi-agent system** — specialization (Nutrition vs Fitness);
- **FIPA ACL** — стандартизирана agent communication;
- **Hybrid persistence** — ontology + relational DB.

2.2. Цели и mapping към заданието

Изискване от заданието	Реализация
GUI приложение	Web UI — Thymeleaf + Bootstrap 5
Онтологии	healthy_lifestyle.owl + Apache Jena
≥ 2 типа агенти	NutritionAgent, FitnessAgent

ACL комуникация	JADE, FIPA REQUEST / INFORM / FAILURE
Манипулация на онтологии	Runtime add custom meals, Person, habit links
База данни	H2 — profiles, logs, consultations, ACL audit
Документация	Настоящ документ

2.3. Обхват

В обхват: nutrition/fitness/habits/general consultations, daily logs с history в UI, ontology manipulation (custom meals), ACL audit, персонализация по profile + logs + query.

Извън обхват: medical diagnosis, wearables, LLM/chatbot, authentication. Прототип за academic demo — **не е свободен чат като ChatGPT**, а structured agent consultation.

2.4. Връзка с дисциплините

Дисциплина	Принос
Интелигентни системи	Multi-agent, ACL, coordinator pattern, SPARQL reasoning
Онтологично инженерство	OWL classes/properties, individuals, runtime manipulation
Интелигентни архитектури	Layered Spring Boot design, separation of concerns
Практикум	End-to-end working application, testing, demo

3. Анализ на предметната област

3.1. Ключови понятия

- **Person** — потребител с goal и activity level
- **Goal** — WeightLoss, MuscleGain, Maintenance, Endurance
- **Meal / Food / Nutrient** — хранителна йерархия
- **Exercise** — Cardio, Strength, Flexibility упражнения
- **Habit** — ежедневни навици (вода, сън, стъпки)
- **WorkoutPlan** — план от препоръчани упражнения

3.2. Функционални изисквания

ID	Изискване
FR-1	CRUD профил + OWL Person sync
FR-2	Nutrition consultation via ACL + SPARQL
FR-3	Fitness consultation via ACL + SPARQL
FR-4	Habits list om ontology (Coordinator)

FR-5	Daily log (вода, стъпки, сън) + history в UI
FR-6	Runtime ontology manipulation (custom meals)
FR-7	ACL message audit trail
FR-8	GENERAL consult — combined lifestyle summary
FR-9	Personalized tips (BMI, logs, query filtering)

3.3. Онтология vs база данни

Данни	Съхранение	Причина
Foods, meals, exercises, goals	OWL	Static knowledge, SPARQL inference
Person + goal links	OWL (runtime)	Semantic reasoning
Username, weight, age	H2	PII, frequent updates
Daily logs	H2	Time-series data
ACL messages	H2	Operational audit, demo proof

4. Онтологичен модел

Namespace: http://www.semanticweb.org/hla/healthy-lifestyle#
Файл: src/main/resources/ontology/healthy_lifestyle.owl (**v4** — пълна OWL DL таксономия)
Reasoner: Apache Jena OWL_MEM_MICRO_RULE_INF
Обем (v4): 45+ OWL класа, 50+ owl:Restriction, 6 inverse property двойки, 16 храни, 8 ястия, 9 упражнения — **840 base / ~1419 inferred triples**.
Секцията по-долу е директен отговор на изискванията за: класова йерархия, OWL restrictions, equivalent/disjoint класове, inverse/characteristic properties, individuals и defined classes.

4.1. Класова йерархия (задълбочена таксономия)

```
Goal → WeightManagementGoal | PerformanceGoal
Food → PlantBasedFood | AnimalProteinFood | DairyFood
     → HighProteinFood | LowCalorieFood | HighFiberFood |
LeanProteinFood (defined)
Meal → BreakfastMeal | LunchMeal | DinnerMeal | SnackMeal |
BalancedMeal
     → PostWorkoutMeal | VegetarianMeal (defined)
Exercise → CardioExercise | StrengthExercise |
FlexibilityExercise
         → HighIntensityExercise | LowIntensityExercise
(defined)
Nutrient → Macronutrient | Micronutrient → Vitamin | Mineral
Habit → HydrationHabit | SleepHabit | ActivityHabit |
MindfulnessHabit
```

```
WorkoutPlan → EnduranceWorkoutPlan
Person, ActivityLevel, MealType, DietaryRestriction,
IntensityLevel
```

4.2. OWL Restrictions (пълен набор)

Тип restriction	Пример в онтологията
someValuesFrom	HighProteinFood ≡ Food ∧ (proteinGrams some ≥15); AnimalProteinFood (providesNutrient some Protein)
allValuesFrom	VegetarianMeal — всички containsFood са PlantBasedFood; DairyFood — providesNutrient all Macronutrient
minCardinality	Person — tracksHabit min 1; Meal — containsFood min 1 (qualified)
maxCardinality	SnackMeal — containsFood max 3; HighIntensityExercise — durationMinutes max 1
exactCardinality	Person — hasGoal exactly 1, hasActivityLevel exactly 1, username exactly 1

4.3. Equivalent Classes (Defined Classes за reasoner)

Defined class	Дефиниция (equivalentClass)
HighProteinFood	Food ∩ (proteinGrams ≥ 15) → reasoner класифицира ChickenBreast, Salmon, Tuna
LowCalorieFood	Food ∩ (calories ≤ 120) → Broccoli, Spinach, Tuna
HighFiberFood	Food ∩ (fiberGrams ≥ 5) → LentilSoup, Avocado
LeanProteinFood	HighProteinFood ∩ LowCalorieFood → **Tuna inferred автоматично**
BreakfastMeal / LunchMeal / DinnerMeal	Meal ∩ hasMealType hasValue Breakfast/Lunch/Dinner

4.4. Disjoint Classes

Disjoint двойки / групи
CardioExercise ⊥ StrengthExercise ⊥ FlexibilityExercise
Macronutrient ⊥ Micronutrient; Vitamin ⊥ Mineral
PlantBasedFood ⊥ AnimalProteinFood
WeightManagementGoal ⊥ PerformanceGoal

BreakfastMeal ⊥ LunchMeal ⊥ DinnerMeal ⊥ SnackMeal
HighIntensityExercise ⊥ LowIntensityExercise
HydrationHabit ⊥ SleepHabit; ActivityHabit ⊥ MindfulnessHabit

4.5. Properties — inverse и характеристики

Inverse property двойки:

Property	Inverse
containsFood	isContainedIn
providesNutrient	nutrientSourceFood
hasGoal	goalHeldBy
recommendedExercise	exerciseInPlan
supportsGoal	supportedByHabit
consumes	consumedBy

Property characteristics (където е семантично обосновано):

Property	Characteristic
hasGoal, hasActivityLevel, hasMealType, username, goalName, habitName, recommendedForActivityLevel	**Functional**
goalHeldBy	**InverseFunctional**
complementsExercise	**Symmetric**
moreIntenseThan	**Asymmetric**, **Transitive**, **Irreflexive**
includesExercise	**Transitive** (subPropertyOf recommendedExercise)
sameGoalCategory	**Symmetric**, **Reflexive**

4.6. Individuals и връзки

- **16 храни** (типизирани: PlantBasedFood, AnimalProteinFood, DairyFood) с providesNutrient, compatibleWithRestriction, isContainedIn
- **8 ястия** с containsFood, mealSuitableForGoal, hasMealType; VegetarianMeal inferred за растителни ястия
- **9 упражнения** с hasIntensityLevel, complementsExercise (напр. Running ↔ Yoga)
- **4 цели** (типизирани WeightManagementGoal / PerformanceGoal) с sameGoalCategory
- **2 Person** (DemoUser, WeightLossUser) с consumes, tracksHabit, hasGoal
- **Nutrients** с inverse nutrientSourceFood (Iron ← Spinach, VitaminC ← Apple)
- **IntensityLevel** с transitive moreIntenseThan: Low ← Moderate ← High

4.7. SPARQL examples

NutritionAgent — meals for weight loss with low-calorie food constraint:

```
PREFIX hla: <http://www.semanticweb.org/hla/healthy-
lifestyle#>
SELECT DISTINCT ?meal ?label WHERE {
  ?meal a hla:Meal ;
        rdfs:label ?label ;
        hla:mealSuitableForGoal hla:WeightLoss .
  ?meal hla:containsFood ?food .
  { ?food a hla:LowCalorieFood } UNION { ?food
hla:calories ?cal . FILTER(?cal <= 120) }
}
```

Muscle gain — meals containing inferred HighProteinFood:

```
?meal hla:containsFood ?food .
?food a hla:HighProteinFood .
```

Inferred defined class — LeanProteinFood:

```
PREFIX hla: <http://www.semanticweb.org/hla/healthy-
lifestyle#>
ASK { hla:Tuna a hla:LeanProteinFood }   # true след OWL
inference
```

Inverse property — храни в ястие:

```
?food hla:isContainedIn ?meal .
?meal hla:containsFood ?food .   # еквивалентно през inverse
```

FitnessAgent — exercises with duration (unchanged).

WorkoutPlan — plans linked to goal:

```
?plan a hla:WorkoutPlan ;
      hla:planSuitableForGoal hla:WeightLoss .
```

4.8. Runtime manipulation (OntologyService)

Операция	Метод	Trigger
Add Person	addPersonInstance()	Save Profile
Add Custom Meal	addCustomFoodWithMeal()	Add Custom Meal form
Link Habit	linkPersonToHabit()	REST API

При **Add Custom Meal** се създават Food + Meal; OWL reasoner класифицира Food като HighProteinFood / LowCalorieFood според стойностите. UI показва base + inferred statement count.

След всяка промяна: infModel.rebind().

5. Multi-Agent архитектура

5.1. Agent types

Agent	Local name	Роля
NutritionAgent	nutrition-agent	Meal SPARQL — **specialist**
FitnessAgent	fitness-agent	Exercise SPARQL — **specialist**

CoordinatorAgent	coordinator	Routing; HABITS; GENERAL summary
GatewayAgent	gateway-agent	Spring ↔ JADE via AgentRequestQueue

5.2. FIPA ACL

Performative	Смисъл
REQUEST	Заявка за консултация
INFORM	Успешен отговор
FAILURE	Грешка при processing

Request content:

```
{
  "username": "demo",
  "goal": "WEIGHT_LOSS",
  "activityLevel": "MODERATE",
  "type": "NUTRITION",
  "query": "light lunch"
}
```

5.3. ACL sequence

```
[1] Web UI → AgentGateway (creates ACL REQUEST +
conversationId)
[2] AgentRequestQueue.enqueue()
[3] GatewayAgent → AchieveREInitiator → CoordinatorAgent
[REQUEST]
[4] CoordinatorAgent:
      NUTRITION → NutritionAgent [REQUEST]
      FITNESS   → FitnessAgent [REQUEST]
      HABITS    → PersonalizedAdviceService + ontology
      GENERAL   → combined meal + exercise + habit
(Coordinator)
[5] Specialist / Coordinator → SPARQL + profile/log context
→ ACL INFORM
[6] Coordinator → ACL INFORM → GatewayAgent
[7] AgentRequestQueue.complete() → UI response
[8] Persist in agent_messages + consultation_requests
```

Coordinator pattern: UI не знае agent topology; single routing point; extensible за нов specialists.

6. Software архитектура

6.1. Layers

```
Presentation  → WebController, ApiController, index.html
Service       → LifestyleService, AgentAuditService,
PersonalizedAdviceService
Agent Layer   → JADE agents, AgentGateway, AgentRequestQueue
Ontology      → OntologyService (Apache Jena)
Persistence   → Spring Data JPA + H2
```

6.2. Technology stack

Java 17 · Spring Boot 3.2 · JADE 4.6 · Apache Jena 4.10 · H2 · Thymeleaf · Bootstrap 5 · Maven

6.3. Ключови класове

Клас	Package	Отговорност
HealthyLifestyleApplication	bg.pu.hla	Spring Boot entry
OntologyService	ontology	OWL, SPARQL, manipulation
PersonalizedAdviceService	service	Profile + logs + query → dynamic advice
LifestyleService	service	Business orchestration
AgentGateway	agent	Spring → JADE entry
AgentRequestQueue	agent	Thread-safe Spring-JADE bridge
GatewayAgent	agent	Queue poll, ACL to Coordinator
CoordinatorAgent	agent	ACL routing
NutritionAgent	agent	Meal reasoning
FitnessAgent	agent	Exercise reasoning
AclMessageFactory	agent	FIPA message builder
JadeBootstrap	agent	Start JADE on app ready
WebController	web	HTML UI
ApiController	web	REST JSON API

6.4. Spring ↔ JADE integration

JADE agents не са Spring beans. SpringContextHolder дава достъп до OntologyService и repositories от agent threads. AgentAuditService е @Transactional за DB writes от agents.

7. База данни

Engine: H2 embedded, file: ./data/hla-db
Console: http://localhost:8080/h2-console (user: sa)

Таблица	Ключови полета	Purpose
user_profiles	username, goal, activity_level, age, weight_kg	User data
daily_logs	log_date, water_ml, steps, sleep_hours	Daily tracking

consultation_requests	type, user_query, agent_response	Consultation history
agent_messages	sender_agent, receiver_agent, performative, content	**ACL audit proof**

8. Потребителски интерфейс и сценарии

8.1. Web UI sections

1. **Active user indicator** — header показва текущ username и goal
2. **User Profile** — save → H2 + OWL Person; формата се pre-fill-ва от DB
3. **Daily Log** — water, steps, sleep → H2; **Log History** таблица под формата
4. **Agent Consultation** — NUTRITION / FITNESS / HABITS / GENERAL → ACL; показва context (BMI, logs), tips, recommendation cards
5. **Ontology Manipulation** — Add Custom Meal → Food + Meal в ontology; влияе на NUTRITION advice

URL: http://localhost:8080/?user=demo · Demo user: demo (създава се автоматично при първи старт)

8.2. Use cases

UC-1 Profile: Form → LifestyleService → DB + addPersonInstance().
UC-2 Nutrition: Ask Agents → ACL chain → SPARQL meals → PersonalizedAdviceService (profile, logs, query) → UI.
UC-3 Fitness: Same flow → exercises + activity-based frequency + log tips.
UC-4 Habits: Coordinator → filtered habits + personalized tips from daily log.
UC-5 Add custom meal: Form → addCustomFoodWithMeal() → statement count ↑ → meal се появява при NUTRITION consult за user goal.
UC-6 GENERAL: Coordinator → top meal + exercise + habit + lifestyle summary.
UC-7 Daily log: Log Today → H2 → следващ consult включва water/steps/sleep в context и tips.

8.3. Installation

```
cd healthy-lifestyle-assistant
mvn spring-boot:run
```

8.4. Примерен agent response (NUTRITION, goal WEIGHT_LOSS, с daily log)

Context (UI info box):
Personal context for demo: BMI 23.7. Goal WEIGHT_LOSS. Latest log: 1000 ml water, 3000 steps, 6.0 h sleep.
Tips:

- Water intake is low (1000 ml) — aim for at least 2000 ml.
- Steps are below 6000 — a 20-minute walk would help today.

Recommended meals (cards): Лека обяд (277 kcal); Боб чорба (280 kcal) — ако е добавена като custom meal.

```
Nutrition Agent Analysis
User: demo | Goal: WEIGHT_LOSS
```

```
Query focus: low calorie
Recommended meals: ...
Tips: ...
```

8.5. Project structure

```
healthy-lifestyle-assistant/
├── pom.xml
├── lib/jade-4.6.0.jar
├── docs/DOCUMENTATION.md
└── src/main/
    ├── java/bg/pu/hla/
    │   ├── agent/       # JADE agents, ACL, gateway
    │   ├── ontology/    # OntologyService
    │   ├── service/     # LifestyleService
    │   ├── domain/      # JPA entities
    │   └── web/         # Controllers
    └── resources/
        ├── ontology/healthy_lifestyle.owl
        └── templates/index.html
```

9. Тестване

ID	Test	Expected
T-01	App start	4 agents in logs
T-02	GET /	Dashboard loads
T-03	Save profile	H2 record
T-04	NUTRITION consult	Meal response + ACL log
T-05	FITNESS consult	Exercise response
T-06	HABITS consult	Filtered habits + tips
T-07	GENERAL consult	Combined summary
T-08	Add custom meal	Statement count ↑; NUTRITION shows it
T-09	Daily log + consult	Context/tips mention log
T-10	H2 agent_messages	REQUEST + INFORM rows

API test:

```
curl -X POST http://localhost:8080/api/users/demo/consult \
  -H 'Content-Type: application/json' \
  -d '{"type":"NUTRITION","query":"low calorie"}'

curl -X POST http://localhost:8080/api/users/demo/ontology/
foods \
  -H 'Content-Type: application/json' \
  -d '{"name":"Боб чорба","calories":280,"protein":12}'
```

10. Заключение

HLA покрива всички изисквания на заданието: web GUI, OWL ontology с SPARQL reasoning и runtime manipulation, два specialist agent types, FIPA ACL с persist audit, H2 database и документация. Архитектурата демонстрира интеграция между intelligent systems, ontological engineering и software architecture.

Limitations: не е medical product; limited food catalog; single-node deployment.

Future: LLM agent, mobile app, PostgreSQL, Protégé visualization.

11. Използвана литература

1. FIPA ACL Message Structure Specification
2. Bellifemine, Caire, Greenwood — *Developing Multi-Agent Systems with JADE*, Wiley
3. Horridge, Bechhofer — *A Practical Guide to Building OWL Ontologies*
4. Apache Jena Documentation — ARQ SPARQL
5. Wooldridge — *An Introduction to MultiAgent Systems*, Wiley

Приложение A: REST API

Method	Endpoint	Description
POST	/api/users	Create/update profile
GET	/api/users/{username}	Get profile
POST	/api/users/{u}/consult	Agent consultation
POST	/api/users/{u}/ ontology/foods	Add custom meal
GET	/api/ontology/habits	List habits
GET	/api/ontology/stats	Statement count
GET	/api/users/{u}/agent-messages	ACL audit